Dry Run Evaluation

Question 1: Greedy Best-First Search

a) Show each step in a table with the node expanded and the open list

Greedy Best-First Search selects the node with the lowest heuristic value \(h(n)\), which estimates the remaining cost to the goal.

The formula is:

f(n)=h(n)

The start node is S, and the goal node is G.

Step	Node expanded	Open list after expansion (node: h)
1	S	B: 4, A: 8
2	B	D: 3, E: 2, A: 8
3	E	G: 0, D: 3, A: 8
4	G	Goal reached

The open list is arranged in ascending order of heuristic value.

b) State the path found and its total cost

The path found is:

S → B → E → G

The total cost is:

$$3 + 6 + 2 = \boxed{11}$$

Therefore, the total path cost is 11.

c) Is the path optimal? Justify your answer

No, the path is not optimal.

Another possible path is:

S → B → D → G

Its total cost is:

$$3 + 2 + 3 = \boxed{8}$$

The Greedy Best-First Search path costs 11, while the alternative path costs 8. Therefore, Greedy Best-First Search does not guarantee the shortest path because it considers only heuristic values and ignores the actual path cost.

Question 2: A* Search

a) Apply A* Search and show the open list

A* Search selects the node with the lowest estimated total cost using:

$$f(n) = g(n) + h(n)$$

Where (g(n) is the actual cost from the start and h(n) is the estimated remaining cost to the goal.

Step	Node expanded	Open list after expansion node: g+h=f
1	S	B: 3+4=7, A: 2+8=10
2	B	D: 5+3=8, A: 2+8=10, E: 9+2=11
3	D	G: 8+0=8, A: 2+8=10, E: 9+2=11
4	G	Goal reached

At each step, the node with the lowest f(n) value is selected. When D is expanded, the path to G has a cost of 8, which is lower than the other open-list values.

b) State the path and its cost, and compare it with Question 1

The path found by A* Search is:

S → B → D → G

Total cost:

$$3 + 2 + 3 = \boxed{8}$$

Greedy Best-First Search found S → B → E → G with a cost of 11. A* finds the shorter route with a cost of 8 because it considers both the distance travelled and the estimated remaining distance.

c) Check whether the heuristic is admissible

A heuristic is admissible if it never overestimates the true shortest remaining cost to the goal.

Node	Heuristic h(n)	True remaining cost	Admissible?
A	8	10	Yes
B	4	5	Yes
D	3	3	Yes
E	2	2	Yes

The heuristic is admissible for all four nodes. This property helps A* find an optimal path when the algorithm is implemented correctly and the other required conditions hold.

Question 3: Constraint Satisfaction Problem (CSP)

a) Formulate the CSP and draw the constraint graph

This is a Constraint Satisfaction Problem because six regions must be assigned colours while ensuring that neighbouring regions have different colours.

Variables: WA, NT, SA, Q, NSW, V.

Domains: Each region can be coloured Red (R), Green (G), or Blue (B).

D(WA)=D(NT)=D(SA)=D(Q)=D(NSW)=D(V)=R,G,B

Constraints:

- WA ≠ NT
- WA ≠ SA
- NT ≠ SA
- NT ≠ Q
- SA ≠ Q
- SA ≠ NSW
- SA ≠ V
- Q ≠ NSW
- NSW ≠ V

Constraint graph:

WA

NT

SA

Q

NSW

V

Regions connected by an edge must have different colours. The constraints listed above define the exact graph connections.

b) Apply backtracking search with forward checking and MRV

The Minimum Remaining Values (MRV) heuristic selects the unassigned region with the fewest legal colours. Ties are broken using the degree heuristic, followed by the specified region order. Colours are tried in the order Red, Green, Blue.

Step	Region selected	Assigned colour	Updated domains of unassigned regions
1	SA	Red (R)	WA: G,B; NT: G,B; Q: G,B; NSW: G,B; V: G,B
2	NT	Green (G)	WA: B; Q: B; NSW: G,B; V: G,B
3	Q	Blue (B)	WA: B; NSW: G; V: G,B
4	NSW	Green (G)	WA: B; V: B
5	WA	Blue (B)	V: B
6	V	Blue (B)	None

Final colouring:

Region	Colour
WA	Blue
NT	Green
SA	Red
Q	Blue
NSW	Green
V	Blue

All neighbouring regions have different colours, so the final colouring satisfies the constraints.

c) Explain how MRV and degree heuristics reduce search effort

The MRV heuristic selects the region with the fewest available colours first. This helps identify difficult choices early and reduces unnecessary attempts.

The degree heuristic breaks ties by choosing the region connected to the largest number of unassigned neighbours. This helps constrain other regions sooner. Forward checking removes colours that become invalid after each assignment.

Together, these techniques reduce the amount of backtracking required.

Question 4: Minimax with a Heuristic Evaluation Function

a) Apply the Minimax algorithm and calculate every node's value

The Minimax algorithm is used in two-player games. MAX tries to obtain the highest score, while MIN tries to obtain the lowest score for MAX.

The leaf values are:

3, 5 | 6, 9 | 1, 2 | 0, -1

Step 1: Calculate the MAX nodes.

MAX selects the highest value from each pair.

Leaf values	MAX value
3, 5	5
6, 9	9
1, 2	2
0, -1	0

Step 2: Calculate the MIN nodes.

MIN selects the lowest value from each pair of MAX-node values.

MAX-node values	MIN value
5, 9	5
2, 0	0

Step 3: Calculate the root MAX node.

The root selects the highest of the two MIN-node values.

$$\max(5,0) = \boxed{5}$$

Therefore, the final value of the root node is 5.

b) State MAX's best first move and the principal variation

MAX chooses the left branch, whose value is 5.

The principal variation, which is the sequence of best moves, is:

Root MAX → Left MIN → Left MAX → Leaf value 5

The final score guaranteed by MAX is 5.

c) Calculate the evaluation function for the tic-tac-toe board

The evaluation function is:

$$E = (\text{open lines for X}) - (\text{open lines for O})$$

The board is:

X	O	Empty
Empty	X	Empty
Empty	Empty	O

A line is open for X if it contains no O. A line is open for O if it contains no X.

- Open lines for X: 3 (second row, first column, and anti-diagonal).
- Open lines for O: 2 (third row and third column).

Therefore,

$$E = 3 - 2 = \boxed{1}$$

The evaluation value is +1, indicating a slight advantage for X according to this evaluation function.

Question 5: Alpha-Beta Pruning

a) Apply Alpha-Beta Pruning from left to right

Alpha-Beta Pruning improves Minimax by skipping branches that cannot affect the final decision.

- MAX tries to maximize the value and updates \(\alpha\).
- MIN tries to minimize the value and updates \(\beta\).
- Pruning occurs when \(\alpha \geq \beta\).

The leaf values from left to right are:

8, 6, 5, 7, 2, 9, 4, 3, 6, 1, 5, 2, 9, 7, 3, 4

Step 1: Evaluate the left subtree.

The first MAX node evaluates the pair (8, 6), giving:

$$\min(8,6) = 6$$

Its next pair is (5, 7). Since the current \(\alpha=6\), evaluating 5 makes \(\beta=5\). As \(\beta \leq \alpha\), the leaf 7 is pruned.

The first MAX node therefore returns 6.

The second MAX node evaluates (2, 9) and (4, 3):

$$\max(2,3) = 3$$

The left MIN node returns:

$$\min(6,3) = 3$$

The root MAX node now has \(\alpha=3\).

Step 2: Evaluate the right subtree.

The first MAX node evaluates (6, 1) and (5, 2).

$$\max(\min(6,1), \min(5,2)) = 2$$

The right MIN node now has beta=2. Since beta=2 eq alpha=3, the remaining MAX subtree containing the leaves (9, 7, 3, 4) is pruned.

Summary of pruning:

Branch	Leaves	Result
First MAX subtree, second MIN pair	5, 7	Leaf 7 pruned
Right MIN subtree, second MAX child	9, 7, 3, 4	All four leaves pruned

The root value is:

\[\max(3,2)=\boxed{3} \]

b) State the number of leaves evaluated and compare with Minimax

Total leaves = 16.

Leaves evaluated:

- Left subtree: 7 leaves.
- Right subtree: 4 leaves.

$$7 + 4 = \boxed{11}$$

Leaves pruned:

$$16 - 11 = \boxed{5}$$

Algorithm	Leaves evaluated	Root value
Plain Minimax	16	3
Alpha-Beta Pruning	11	3

Both algorithms return the same root value of 3. Alpha-Beta Pruning evaluates five fewer leaves.

c) Explain how move ordering changes the amount of pruning

Move ordering affects how many branches can be pruned. If the best moves are examined earlier, stronger alpha and beta bounds are established, allowing more branches to be skipped.

If less promising moves are examined first, fewer branches may be pruned. However, Alpha-Beta Pruning produces the same final result as Minimax when applied correctly; it only reduces the number of positions that need to be evaluated.